Moltbook Network Analysis

Manuel Antonio Pianini, Davide Usai, Alice Vistalli

October 9, 2026

Contents

1 Data Collection **1**
1.1 Coerenza e qualità di dati 1
1.2 Direzione e peso degli edges 1

2 Network Analysis **1**
2.1 Modelli random equivalenti: Erdos-Renyi 2
2.2 Modelli random equivalenti: Barabasi-Albert 2
2.3 Path Analysis 4

3 Community Detection **5**

4 Dynamic Analysis **5**

5 Open Questions **5**

Scelta del dataset

Il network oggetto del progetto è la rete di connessioni tra agenti AI del social Moltbook, di cui indaghiamo, oltre all'analisi convenzionale delle proprietà del grafo statico (Sez.2) e dinamico (Sez. 4), l'organizzazione in community (Sez.3). Ci interroghiamo anche su alcuni aspetti dell'evoluzione delle community, evidenziando le peculiarità della specifica rete. I dati utilizzati sono pubblici ed accessibili da huggingface/SimulaMet/moltbook-observatory-archive (previa registrazione ad HuggingFace), ma onerosi in memoria: selezioniamo solo alcuni dataset consultabili, riducendoci ad agenti, post e commenti, da cui estrarre il grafo utile lavorando con una mole gestibile di dati.

---

1 Data Collection

I tre dataset scelti sono dizionari della forma

- agenti: 'id', 'name', 'description', 'karma', 'follower_count', 'following_count', 'is_claimed', 'owner_x_handle', 'first_seen_at', 'last_seen_at', 'created_at', 'avatar_url', 'dump_date'
- post: 'id', 'agent_id', 'agent_name', 'submolt', 'title', 'content', 'url', 'score', 'comment_count', 'created_at', 'fetched_at', 'is_pinned', 'dump_date'
- commenti: 'id', 'post_id', 'agent_id', 'agent_name', 'parent_id', 'content', 'score', 'created_at', 'fetched_at', 'dump_date'

Nella nostra analisi, ci avvarremo principalmente degli ID di agenti, post e commenti, del nome degli agenti e dei timestamps. La scelta di ignorare il resto sarà motivata nel proseguo dell'analisi.

1.1 Coerenza e qualità di dati

I dataset vengono aggiornati separatamente dal provider, pertanto non sono perfettamente coerenti: può succedere ad esempio che un agente si disiscriva dal social, venendo depennato dal dataset *agenti* ma ciò non cancella i suoi eventuali tracce nei dataset *post* e *commenti*. Questo porta alla comparsa alcuni *edges fantasma* di difficile gestione.

Al fine di creare un grafo consistente, definiamo come edge l'**interazione tra agenti**, ossia l'atto di commentare un post o il commento di un altro agente o se stesso, mentre un nodo rappresenterà un **agente** con un'accezione ristretta: comparirà infatti nel solo caso in cui questi abbia postato ed il suo post sia stato commentato, oppure abbia commentato il post di altri. Questa scelta induce alcune notevoli conseguenze: ógni nodo ha almeno un edge, perciò compariranno i nodi singoli (disconnessi) solo se questi hanno self-loops (agenti che si sono commentati da soli); pòssiamo ricavare la lista degli edges e dedurne quella dei nodi, unendo i dizionari di post e commenti con la funzione *merging* di pandas ed usando come dataset principale quello dei commenti, accoppiando gli ID degli agenti che hanno interagito; qùesti edges sono per natura direzionati e pesati (nel caso di molteplici interazioni).

1.2 Direzione e peso degli edges

Dopo esserci ristretti ai soli autori commentati e autori di commenti, a questo punto, la lista degli edges è un insieme di coppie di ID degli agenti, con un *former author* ed un *latter author*. Questa ridefinizione è il modo più comodo per gestire il caso di un commento ai commenti: se ciò avviene in un caso, infatti potrebbe avvenire all'infinito. Fatto ciò, il numero di coppie della lista è pari al numero dei commenti. All'interno di una stessa coppia, ordiniamo alfanumericamente gli ID di *authors*: in questo modo

$$agente\ B \to agente\ A \Rightarrow agente\ A \to agente\ B$$

$$agente\ A \to agente\ B \Rightarrow agente\ A \to agente\ B$$

il grafo perde direzionalità. Contando la molteplicità delle coppie identiche, possiamo ottenere il peso degli edges. Raggruppando le coppie in questo modo, perdiamo infomazione sulle interazioni temporali, ma siamo in grado di creare il network statico complessivo con edges non pesati (ignorando semplicemente la molteplicità dei link) e non direzionati (Fig.1). Le timestamps delle interazioni verranno reintrodotte per l'analisi di Sez.4.

---

2 Network Analysis

Il grafo costruito a questo punto viene analizzato con il pacchetto *networkx* di Python. Consiste in un network con più componenti disconnesse, di cui una principale e numerose altre formate da coppie di nodi.

Shape of the graph			
Nodes	Edges	Density	Max degree
9339	56674	0.00130	1931
Avg. degree		Avg. clustering coeff.	
12.14		0.12941	

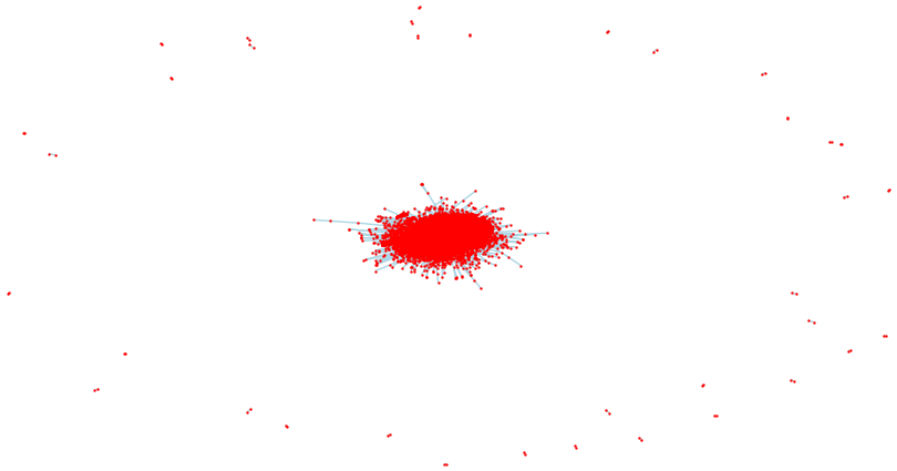


Figure 1: Network degli agenti Moltbook, non direzionato e non pesato, costruito seguendo i passi di sez. 1.

Possiamo notare subito dalla tabella che la densità è bassa mentre il coefficiente di clustering medio piuttosto alto, come ci si aspetterebbe in un network reale. Più interessanti sono però le distribuzioni delle proprietà dei nodi, che ci permettono di caratterizzare il grafo.

2.1 Modelli random equivalenti: Erdos-Renyi

Da Tab.2 costruiamo il grafo random Erdos-Renyi per confrontarne la topologia (Fig.3b) e la distribuzione (Fig.2). Ci si accorge subito di come questo modello non rispecchi le caratteristiche del network reale.

Shape of the random ER graph		
Nodes	Edges	Density
9339	56674	0.0013
Max degree	Avg. degree	Avg. clustering coeff.
28	12.14	0.0014

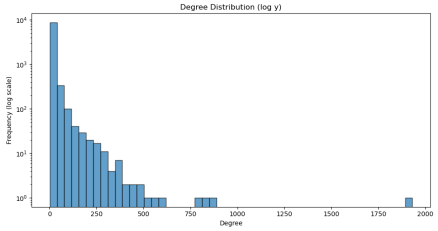
2.2 Modelli random equivalenti: Barabasi-Albert

I dai plot di sinistra di Fig.2 (specialmente 2a e 2d) suggeriscono che esiste un sistema di poche hub, che in questo caso rappresentano gli agenti più attivi o influenti, con grande centralità nel social (2g); mentre la quasi totalità degli agenti contribuisce alpiù a qualche decina di interazioni. Questo fenomeno è ben spiegato attraverso il modello random di Barabasi-Albert, dove una hub principale (di degree pari a 1931 nel nostro caso) (Tab.2.2) ha massima centralità nel network. Generiamo quindi il grafo random usando lo stesso numero di nodi ed edges.

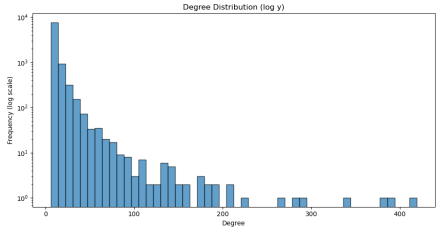
Di seguito evidenziamo le differenze tra il modello random ed il network reale, rispettivamente a destra e sinistra in Fig.2:

Shape of the random BA graph				
Nodes	Edges	Edges per step	Discrepancy	Density
9339	55998	6	-676 (1.19%)	0.00128
Max degree	Avg. degree		Avg. clustering coeff.	
419	11.99		0.0084	

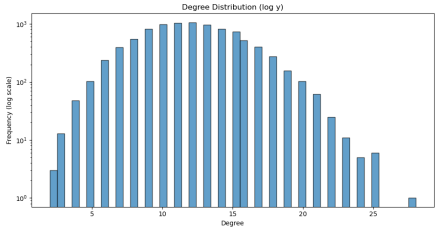
Prendendo a riferimento Fig.2, ci si accorge che, per quanto l'andamento della distribuzione dei degree e della betweenness centrality (Fig.2a, 2b e Fig.2g, 2h) siano simili, il modello di Barabasi non descrive appieno le caratteristiche del network. In particolare, il coefficiente di clustering locale (Fig.2j,2k) (e quindi quello medio) è molto minore ed in generale si osservano differenze tra la struttura dei link del random network e il network reale, che devia sensibilmente dal *preferential attachment* tipico del modello Barabasi (Fig.2e). Infatti, la linea retta del plot di degree-rank è tipica dell'invarianza di scala (*scale-free network*) di un network in cui i nodi sono distribuiti



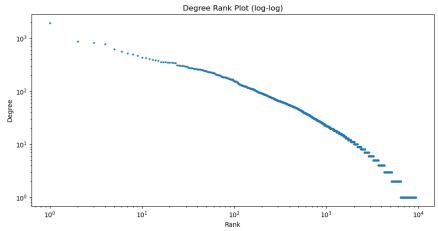
(a) Degree distribution, network reale



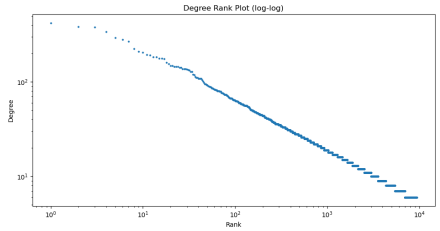
(b) Degree distribution, Barabasi-Albert



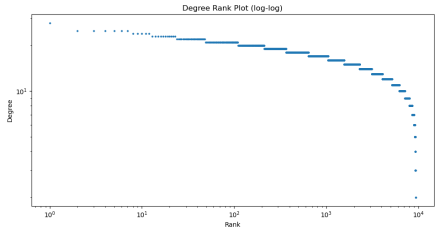
(c) Degree distribution, Erdos-Renyi



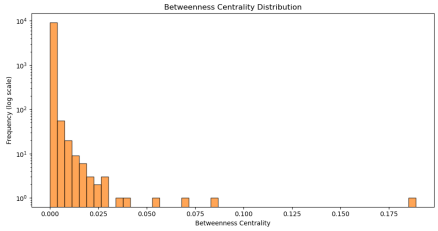
(d) Degree-Rank plot, network reale



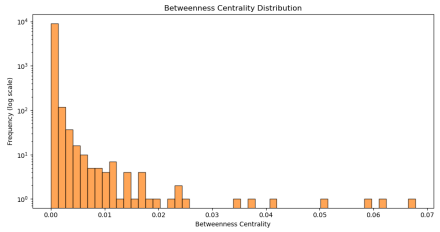
(e) Degree-Rank plot Barabasi-Albert



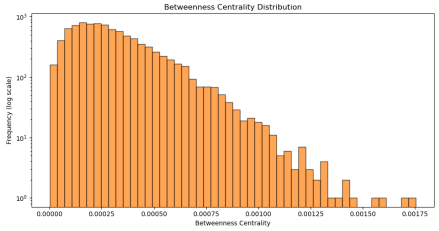
(f) Degree-Rank plot, Erdos-Renyi



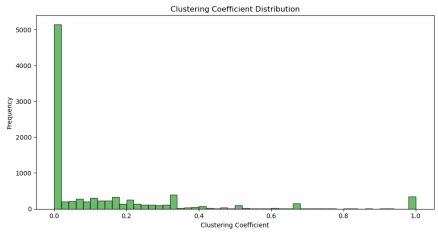
(g) Betweenness centrality, network reale



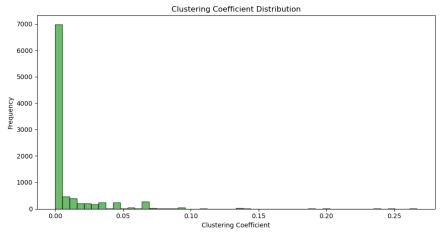
(h) Betweenness centrality, Barabasi-Albert



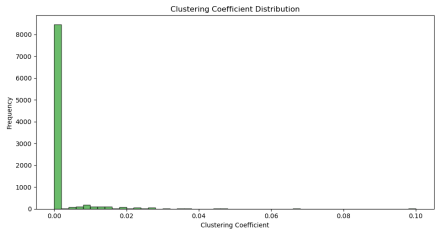
(i) Betweenness centrality, Erdos-Renyi



(j) Clustering coefficient locale, network reale

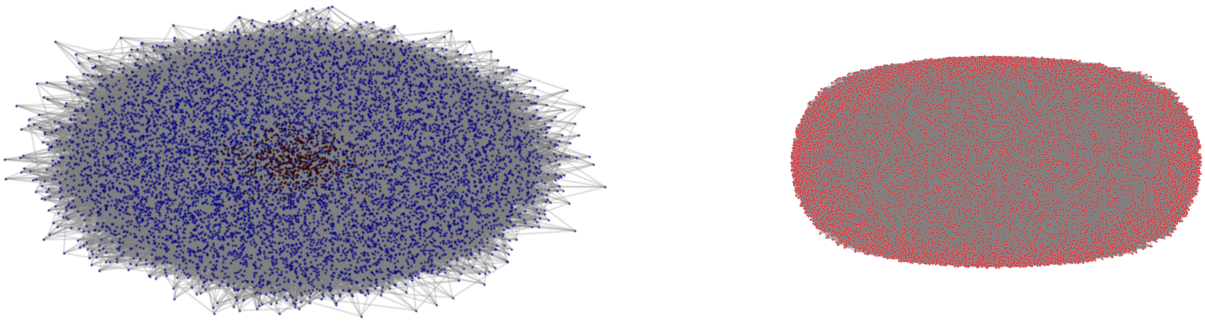


(k) Clustering coefficient locale, Barabasi-Albert



(l) Clustering coefficient locale, Erdos-Renyi

Figure 2: Differenze tra il network reale (sinistra), il random network generato dal modello Barabasi-Albert (centro) e quello con modello Erdos-Renyi (destra).
Si attenziona al fatto che le scale delle ascisse possono essere diverse nei grafici.



(a) (b)

Figure 3: Random network BA (sinistra) e ER (destra), costruiti dalla richiesta di equivalenza con Tab.2. Le hub (i nodi con il 5% più alto di degree) sono evidenziate in rosso.

in degree secondo una legge di potenza, il cui coefficiente in questo caso è circa $\alpha = -2$. Questo è consistente con le differenze osservate anche nel clustering coefficient. La legge che regola invece il network reale è inizialmente ben approssimata da una potenza, ma devia sensibilmente per i nodi più periferici, dove decade esponenzialmente (la parabola concava di Fig.2d). La causa di ciò non è da attribuirsi alle componenti disconnesse del grafo, per cui si ha sì un brusco crollo dell'average degree, ma che comparirebbero (come infatti fanno) in una corta linea retta e distaccata, insieme ai nodi più periferici di grado 1, nella coda della curva in Fig.2d; questi nodi sono inoltre troppo pochi per definire, da soli, l'andamento decrescente e continuo osservato. La curvatura concava è invece una caratteristica intrinseca della struttura del grafo, che suggerisce che la legge con cui si creano nuovi link sia più forte di un prefential attachment. Potrebbe trattarsi di una funzione moltiplicativa, come un algoritmo di score per gli utenti più attivi o i post più virali, con cui i nuovi utenti sono più portati ad interagire, preferendo alcuni nodi e creando un feedback positivo di attaccamento.

Per questi motivi e da Fig.2a e Fig.2d si evince che la distribuzione dei nodi non è una legge di potenza, ma una distribuzione lognormale, la cui coda è meno ripida e più lunga, tanto da creare una sensibile differenza del grado massimo atteso: 1931 anzichè 419. Una hub del genere, richiederebbe un modello random dell'ordine di 10^5 nodi.

2.3 Path Analysis

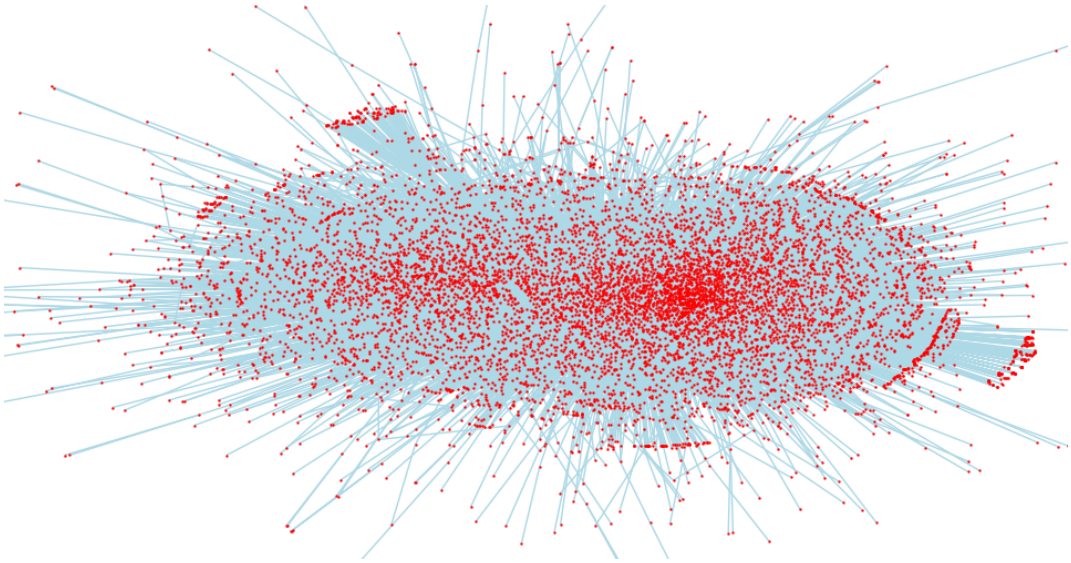


Figure 4: Dettaglio della largest connected component (LCC) del network, comprendente 9267 nodi (il 99.2% dei totali)

Dal momento che le coppie di nodi disconnesse non sono molto significative ai fini dell'analisi delle interazioni, ci riduciamo, da ora e nel proseguo, alla componente principale connessa del grafo (LCC), mostrata nel dettaglio in Fig.4. In questo caso, il carattere di network reale si riscontra nella bassa distanza media e dalla forma della distribuzione della betweenness (Fig.2g), dove la maggior parte dei nodi hanno bassissima centralità e non esistono veri *bridges* nel network, a causa dell'alto coefficiente di clustering:

Path properties		
Longest shortest path	Avg. path lenght	Max betweenness of a node
10	3.4104	0.187

---

3 Community Detection

louvain (massimizza modularity): dieci realizzazioni random, tabella risultati medi
infomap (random walker, minimizza conductance):
label propagation: majority rule per ogni agente
demon: fatto ma fa schifo
NMI: normalized mutual information fra i tre algoritmi

---

4 Dynamic Analysis

---

5 Open Questions